

Adicionando *MCP do GitHub*

O servidor MCP oficial do GitHub deixa o agente do Cursor ler repositórios, cuidar de issues e pull requests e acompanhar o GitHub Actions, tudo pelo chat. Nesta aula: os três jeitos de instalar, como criar um token com o mínimo de permissão, como escolher os conjuntos de ferramentas, como testar e como usar com segurança.

`github/github-mcp-server`

Marketplace · `/add-plugin github`

Remoto com PAT

Local com Docker

Toolsets · `read-only`

`${env:...}`

Formação Claude Code 2026 · Baseado em github.com/github/github-mcp-server (README, guia Install in Cursor, Remote Server), cursor.com/marketplace/cursor/github e cursor.com/docs/mcp, consultados em outubro de 2026 · Telas: reprodução ilustrativa

O que o servidor *do GitHub* faz

O **GitHub MCP Server** conecta ferramentas de IA direto à plataforma do GitHub: ler repositórios e arquivos, gerenciar issues e PRs, analisar código e automatizar fluxos, tudo em linguagem natural.

USO	EXEMPLOS
Repositórios	Buscar código e arquivos, analisar commits, entender a estrutura
Issues e PRs	Criar, atualizar, triar bugs, revisar mudanças
CI/CD	Acompanhar o GitHub Actions, analisar falhas de build
Segurança	Alertas do Dependabot e do code scanning
Time	Discussões, notificações, atividade

MCP DO GITHUB × GIT NO TERMINAL

O agente já sabe usar o `git` no terminal para o repositório **local** (commit, branch, diff). O MCP do GitHub fala com o **github.com**: issues, PRs, Actions, código de outros repositórios. Os dois se completam.

O pacote npm antigo `@modelcontextprotocol/server-github` **não é mais suportado** desde abril de 2025. Tutoriais antigos ainda o usam; ignore-os.

Três jeitos *de instalar*

CAMINHO	ONDE RODA	AUTENTICAÇÃO	PRECISA DE	PARA QUEM
A · Marketplace do Cursor	Plugin GitHub criado e verificado pelo Cursor	A que o fluxo de instalação pedir	Login no Cursor	Quem quer o caminho mais simples
B · Remoto (recomendado pelo GitHub)	Servidor hospedado pelo GitHub: <code>api.githubcopilot.com/mcp/</code>	Personal Access Token no cabeçalho	Um PAT	Quem quer controlar o token e os toolsets
C · Local com Docker	Na sua máquina, imagem <code>ghcr.io/github/github-mcp-server</code>	OAuth no navegador ou PAT	Docker rodando	Quem prefere tudo local ou usa GitHub Enterprise

O guia do GitHub para o Cursor diz que, embora o Cursor suporte OAuth em alguns servidores, o servidor **remoto** do GitHub **pede um PAT** nesse caminho manual.

Escolha **um** caminho. Dois servidores do GitHub ao mesmo tempo duplicam as ferramentas e confundem o agente.

Pelo *Marketplace*

- 1 Abra **Customize** › **MCPs** (ou a aba de plugins) e busque **GitHub**.
- 2 Confira o selo: **Created by Cursor** · **Verified by Cursor**.
- 3 Clique em **Add to Cursor**. Atalho no chat: `/add-plugin github`.
- 4 Siga a autenticação com a sua conta do GitHub e autorize só o que precisar.
- 5 Verifique o ponto verde e as ferramentas, como na seção 08.

PLUGIN, NÃO SÓ SERVIDOR

No Marketplace, o GitHub vem como **plugin**: um pacote que pode trazer servidor MCP, rules, skills e comandos juntos. Na página dele aparece "MCP 1": o servidor `github`.

As telas do Marketplace mudam com frequência. Se algo estiver diferente, a referência é a página `cursor.com/marketplace/cursor/github`.

Criando o *token (PAT)*

- 1

No GitHub: **Settings** > **Developer settings** > **Personal access tokens** > **Fine-grained tokens** (ou direto em `github.com/settings/personal-access-tokens/new`).
- 2

Dê um nome claro ("*cursor-mcp*") e uma **validade** curta, como 30 ou 90 dias.
- 3

Em **Repository access**, escolha **Only select repositories** e marque só os do curso.
- 4

Em **Permissions**, libere só o que vai usar (tabela ao lado).
- 5

Gere e copie o token na hora: o GitHub não mostra de novo.

PERMISSÃO DO REPOSITÓRIO	PARA
Metadata · leitura	Obrigatória (já vem marcada)
Contents · leitura	Ler arquivos e código
Issues · leitura e escrita	Listar, criar e comentar issues
Pull requests · leitura e escrita	Listar, criar e revisar PRs
Actions · leitura	Ver execuções e logs de CI

O README do servidor avisa: ele usa muitas APIs do GitHub, então libere só as **permissões com que você se sente à vontade** em dar a uma IA. Para só consultar, use tudo em **leitura**.

Configurando *o servidor remoto*

~/CURSOR/MCP.JSON · COMO NO GUIA DO GITHUB

```
{
  "mcpServers": {
    "github": {
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "Bearer YOUR_GITHUB_PAT"
      }
    }
  }
}
```

MELHOR: O TOKEN NUMA VARIÁVEL DE AMBIENTE

```
"headers": {
  "Authorization": "Bearer ${env:GITHUB_PAT}"
}
```

- 1 Abra (ou crie) `~/cursor/mcp.json`, o arquivo **global**.
- 2 Cole a configuração. Use `${env:GITHUB_PAT}` e defina a variável no sistema.
- 3 Salve e reinicie o **Cursor** por completo.

O guia também oferece um botão de instalação que preenche isso; depois é só trocar `YOUR_GITHUB_PAT` pelo token. O transporte é Streamable HTTP (Cursor 0.48 ou mais novo).

Prefira o arquivo **global**. O `.cursor/mcp.json` do projeto costuma ir para o Git, e um token ali vaza junto.

Definindo *GITHUB_PAT* em cada sistema

SISTEMA	COMO DEFINIR (PERMANENTE PARA O SEU USUÁRIO)
Windows	PowerShell: <code>setx GITHUB_PAT "seu_token"</code> · ou Configurações > Sistema > Sobre > Configurações avançadas > Variáveis de Ambiente
macOS	No <code>~/ .zshrc</code> : <code>export GITHUB_PAT="seu_token"</code> · abra o Cursor a partir de um terminal novo, ou reinicie a sessão
Linux	No <code>~/ .bashrc</code> ou <code>~/ .zshrc</code> : <code>export GITHUB_PAT="seu_token"</code> · faça logout e login para valer em apps abertos pelo menu

A ajuda do Cursor avisa: se o servidor depende de variáveis definidas no perfil do shell, elas precisam estar disponíveis **para o Cursor**. Depois de mudar o perfil, **reinicie o Cursor**.

Para servidores **locais** (stdio), dá também para usar `"envFile": ".env"` . Servidores remotos não aceitam `envFile` : use `${env:...}` .

Servidor *local com Docker*

COM LOGIN OAUTH (SEM TOKEN PARA GUARDAR)

```
{
  "mcpServers": {
    "github": {
      "command": "docker",
      "args": ["run", "-i", "--rm",
        "-p", "127.0.0.1:8085:8085",
        "-e", "GITHUB_OAUTH_CALLBACK_PORT",
        "ghcr.io/github/github-mcp-server"],
      "env": { "GITHUB_OAUTH_CALLBACK_PORT": "8085" }
    }
  }
}
```

COMO FUNCIONA

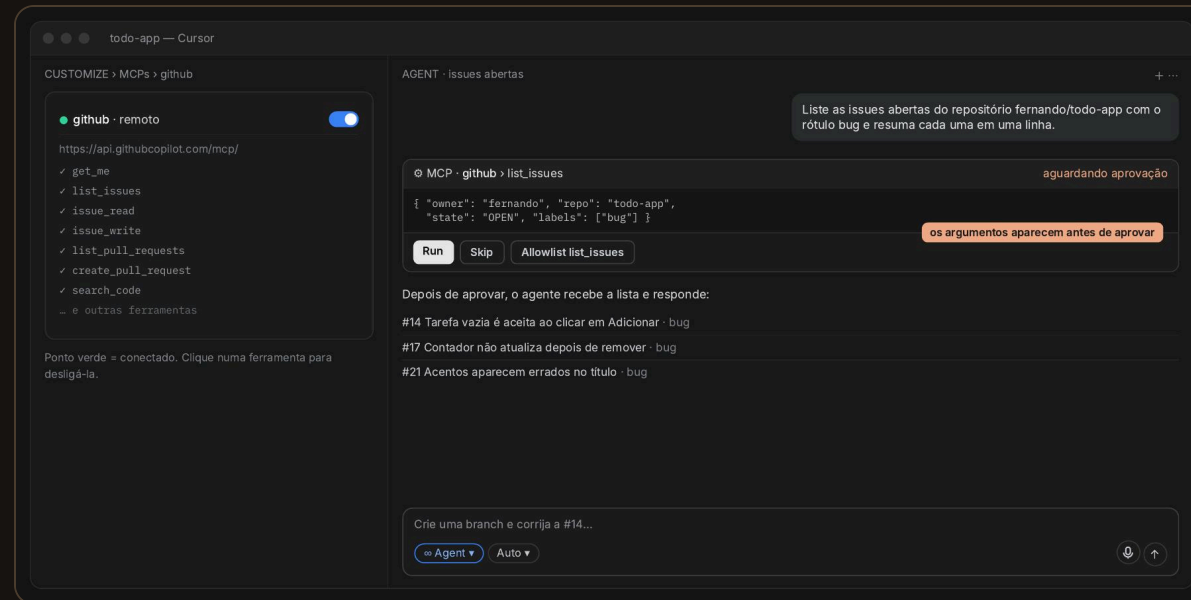
No github.com, a imagem oficial já traz as credenciais do app: no primeiro uso abre um **login no navegador** e o token fica **só na memória**. A porta fixa 8085 recebe o retorno do login.

OU COM PAT (TEM PRIORIDADE SOBRE O OAUTH)

```
"args": ["run", "-i", "--rm", "-e",
  "GITHUB_PERSONAL_ACCESS_TOKEN",
  "ghcr.io/github/github-mcp-server"],
"env": { "GITHUB_PERSONAL_ACCESS_TOKEN":
  "${env:GITHUB_PAT}" }
```

O **Docker Desktop** (ou o Docker Engine, no Linux) precisa estar **rodando**. Erro ao baixar a imagem: tente `docker logout ghcr.io`.

Funcionou? *Teste*



Servidor conectado, chamada aguardando aprovação e resultado · reprodução ilustrativa; repositório e issues de exemplo

1. Reinicie o Cursor e procure o **ponto verde** no servidor `github` .

2. Peça: *"liste os meus repositórios no GitHub"* ou *"quem sou eu no GitHub?"* (`get_me`).

3. Aprove a chamada e confira a resposta. Erro? MCP Logs.

Toolsets *e modo somente leitura*

O servidor tem perto de cem ferramentas, agrupadas em **toolsets**. Sem configuração, vem o conjunto `default` :

- context
- repos
- issues
- pull_requests
- users

NO SERVIDOR REMOTO, USE A URL	RESULTADO
<code>.../mcp/</code>	Toolsets padrão
<code>.../mcp/readonly</code>	Padrão, só ferramentas de leitura
<code>.../mcp/x/actions</code>	Só GitHub Actions
<code>.../mcp/x/issues/readonly</code>	Só issues, só leitura
<code>.../mcp/x/all</code>	Todos os toolsets

OUTROS TOOLSETS

`actions` , `code_security` , `dependabot` , `discussions` , `gists` , `labels` , `notifications` , `orgs` , `projects` , `secret_protection` , `security_advisories` , `stargazers` e outros.

Menos ferramentas = menos contexto ocupado e menos chance de o agente escolher a ferramenta errada. Comece com `readonly` e libere escrita quando precisar.

No servidor **local**, o equivalente é `GITHUB_TOOLSETS="default,actions"` e `GITHUB_READ_ONLY=1` nas variáveis do Docker.

Pedidos que *usam o GitHub*

TRIAGEM

Liste as issues abertas com o rótulo bug em fernando/todo-app e ordene por quantidade de comentários. Resuma cada uma em uma linha.

DA ISSUE AO PR

Leia a issue #14, corrija o problema numa branch nova, rode os testes e abra um pull request que cite a issue. Não faça merge.

CI

A última execução do GitHub Actions falhou. Leia os logs do job que quebrou e me diga a causa provável, sem mudar nada ainda.

REVISÃO

Leia o PR #22 e deixe um comentário de revisão com riscos e sugestões. Não aprove nem feche.

Diga sempre **até onde** o agente pode ir ("não faça merge", "sem mudar nada ainda"). Ações no GitHub são visíveis para o time.

Usando *com segurança*

- ✓ **Token fine-grained**, só nos repositórios necessários e com validade curta
- ✓ **Nunca** deixe o token num arquivo que vai para o Git
- ✓ Comece com **readonly**; libere escrita por toolset
- ✓ Mantenha a **aprovação** das ferramentas de escrita (criar, comentar, fazer merge)
- ✓ Coloque na allowlist só ferramentas de **leitura** que você usa sempre
- ✓ Desconfie de issues e PRs de terceiros: o texto pode tentar dar ordens ao agente (**injeção de prompt**)
- ✓ Revogue o token no GitHub se ele vazar ou não for mais usado
- ✓ Para organizações, veja se há aprovação de tokens pelos admins

Modo lockdown: em repositórios públicos, o servidor só mostra conteúdo (issues, PRs, comentários) de quem tem permissão de push, reduzindo o risco de injeção de prompt. É um filtro de melhor esforço, **não** uma barreira de permissão. Local: `GITHUB_LOCKDOWN_MODE=1` ; remoto: cabeçalho `X-MCP-Lockdown` .

Quando *não conecta*

SEM PONTO VERDE

Reinicie o Cursor **por completo** e valide o JSON (vírgulas, aspas).

401 · NÃO AUTORIZADO

Token errado, expirado ou variável vazia. Teste o token e reinicie o Cursor depois de definir a variável.

403 · SEM PERMISSÃO

Falta a permissão no token ou o repositório não foi selecionado no fine-grained.

FERRAMENTAS NÃO APARECEM

Servidor desligado em Customize, ou toolset sem a ferramenta (por exemplo, `readonly` não tem escrita).

DOCKER

Docker parado ou não instalado. Abra o Docker Desktop e tente de novo.

REDE

Proxy ou firewall da empresa bloqueando `api.githubcopilot.com`. Fale com o time de TI.

Em todos os casos, o primeiro lugar para olhar é **Output > MCP Logs** (`Ctrl` + `Shift` + `U`).

Teste o que *ficou*

1. Qual é a URL do servidor remoto oficial do GitHub?

A) github.com/mcp

B) api.githubcopilot.com/mcp/

C) mcp.github.io

D) localhost:8085

B. Hospedado pelo próprio GitHub.

2. Onde é mais seguro configurar o servidor com token?

A) No .cursor/mcp.json do projeto, com o token escrito

B) No ~/.cursor/mcp.json, com \${env:GITHUB_PAT}

C) No README

D) Numa rule

B. O token fica fora do repositório.

3. Como deixar o servidor remoto só com ferramentas de leitura?

A) Não dá

B) Usar a URL terminada em /readonly

C) Desligar o Cursor

D) Usar o pacote npm antigo

B. Por exemplo, .../mcp/readonly.

4. O que o pacote @modelcontextprotocol/server-github tem de errado?

A) Nada

B) Não é mais suportado desde abril de 2025

C) Só roda no macOS

D) É pago

B. Use o servidor oficial.

O GitHub

dentro do chat

Instale pelo **Marketplace**, pelo servidor **remoto** com um PAT fine-grained ou pelo **Docker** local. Guarde o token numa variável (`${env:GITHUB_PAT}`) no `mcp.json` global, comece com **readonly**, escolha os toolsets e deixe claro até onde o agente pode ir.

Marketplace · remoto · Docker

PAT fine-grained

`${env:GITHUB_PAT}`

`/readonly` · `/x/toolset`

MCP Logs

Próxima aula → 09 · Controlando o Navegador com MCP

Fontes: github.com/github/github-mcp-server (README; docs/installation-guides/install-cursor.md; docs/remote-server.md); cursor.com/marketplace/cursor/github; cursor.com/docs/mcp; cursor.com/help/customization/mcp, consultados em outubro de 2026. Os comandos de variáveis de ambiente seguem o padrão de cada sistema. A tela é uma reprodução ilustrativa.